

NOX — Production Grade Flutter Repository Blueprint

Overview

This document contains the complete production-ready repository structure, configuration strategy, CI/CD setup, architecture standards, and implementation blueprint for the NOX Urban Entertainment Infrastructure Platform.

The structure is optimized for:

- Flutter scalability
- Team collaboration
- Supabase integration
- CI/CD automation
- Production deployment
- Feature isolation
- Long-term maintainability

1. Root Repository Structure

```
nox/
├── .github/
│   └── workflows/
│       ├── flutter_ci.yml
│       ├── android_release.yml
│       └── supabase.yml
├── android/
├── ios/
├── web/
├── assets/
│   ├── fonts/
│   ├── images/
│   └── icons/
├── lib/
│   ├── app/
│   │   ├── bootstrap.dart
│   │   └── nox_app.dart
```

```
|
|
|   └─ router.dart
|
| └─ core/
|   └─ config/
|     └─ env.dart
|     └─ supabase_config.dart
|
|   └─ network/
|     └─ api_client.dart
|     └─ result.dart
|
|   └─ errors/
|     └─ failures.dart
|
|   └─ theme/
|     └─ colors.dart
|     └─ nox_theme.dart
|
|   └─ utils/
|     └─ logger.dart
|     └─ validators.dart
|     └─ formatters.dart
|
|   └─ di/
|     └─ providers.dart
|
| └─ features/
|   └─ auth/
|   └─ events/
|   └─ tickets/
|   └─ payments/
|   └─ pulse/
|   └─ inbox/
|   └─ profile/
|
| └─ shared/
|   └─ widgets/
|   └─ constants/
|   └─ extensions/
|
| └─ main.dart
|
| └─ supabase/
|   └─ migrations/
|   └─ functions/
|   └─ seed.sql
```

```
|
|— integration_test/
|— test/
|
|— .env.dev
|— .env.staging
|— .env.prod
|
|— analysis_options.yaml
|— pubspec.yaml
└─ README.md
```

2. VS Code Recommended Extensions

Install these extensions:

```
Dart
Flutter
Error Lens
GitLens
Better Comments
Flutter Widget Snippets
Pubspec Assist
Awesome Flutter Snippets
REST Client
```

3. VS Code Workspace Settings

Create:

```
.vscode/settings.json
```

```
{
  "dart.flutterSdkPath": ".fvm/flutter_sdk",
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll": true
  },
  "files.exclude": {
```

```
    "**/.dart_tool": true,  
    "**/build": true  
  }  
}
```

4. FVM Setup

Install FVM:

```
dart pub global activate fvm
```

Initialize Flutter version:

```
fvm install 3.22.0  
fvm use 3.22.0
```

5. pubspec.yaml (Production)

```
name: nox_mobile  
  
description: NOX Urban Entertainment Platform  
  
publish_to: 'none'  
  
version: 1.0.0+1  
  
environment:  
  sdk: '>=3.2.0 <4.0.0'  
  
dependencies:  
  flutter:  
    sdk: flutter  
  
  flutter_riverpod: ^2.5.1  
  supabase_flutter: ^2.8.0  
  go_router: ^14.2.0  
  flutter_dotenv: ^5.1.0  
  qr_flutter: ^4.1.0  
  intl: ^0.19.0
```

```
url_launcher: ^6.3.0
freezed_annotation: ^2.4.1
json_annotation: ^4.9.0
dio: ^5.5.0
logger: ^2.4.0

dev_dependencies:
  flutter_test:
    sdk: flutter

  build_runner: ^2.4.12
  freezed: ^2.5.7
  json_serializable: ^6.8.0
  flutter_lints: ^4.0.0

flutter:
  uses-material-design: true

  assets:
    - .env.dev
    - .env.staging
    - .env.prod

  fonts:
    - family: Satoshi
      fonts:
        - asset: assets/fonts/Satoshi-Regular.otf
        - asset: assets/fonts/Satoshi-Bold.otf
```

6. Bootstrap Layer

lib/main.dart

```
import 'app/bootstrap.dart';

void main() async {
  await bootstrap();
}
```

lib/app/bootstrap.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_dotenv/flutter_dotenv.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

import '../core/config/env.dart';
import 'nox_app.dart';

Future<void> bootstrap() async {
  WidgetsFlutterBinding.ensureInitialized();

  await dotenv.load(fileName: '.env.dev');

  await Supabase.initialize(
    url: Env.supabaseUrl,
    anonKey: Env.supabaseAnonKey,
  );

  runApp(const ProviderScope(child: NoxApp()));
}
```

7. Environment Layer

lib/core/config/env.dart

```
import 'package:flutter_dotenv/flutter_dotenv.dart';

class Env {
  static String get supabaseUrl =>
    dotenv.env['SUPABASE_URL']!;

  static String get supabaseAnonKey =>
    dotenv.env['SUPABASE_ANON_KEY']!;
}
```

8. Theme Layer

lib/core/theme/colors.dart

```
import 'package:flutter/material.dart';

const Color noxBlack = Color(0xFF070707);
const Color deepGraphite = Color(0xFF141414);
const Color electricCyan = Color(0xFF00AEEF);
const Color neonViolet = Color(0xFF8B5CF6);
const Color iceWhite = Color(0xFFFF5F7FA);
const Color softGray = Color(0xFFA1A1AA);
const Color emeraldPulse = Color(0xFF00D68F);
```

lib/core/theme/nox_theme.dart

```
import 'package:flutter/material.dart';
import 'colors.dart';

final ThemeData noxTheme = ThemeData.dark().copyWith(
  scaffoldBackgroundColor: noxBlack,
  primaryColor: electricCyan,
  cardColor: deepGraphite,
);
```

9. Router Layer

lib/app/router.dart

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';

import '../features/events/presentation/screens/home_screen.dart';

final routerProvider = Provider<GoRouter>((ref) {
  return GoRouter(
    initialLocation: '/',
    routes: [
      GoRoute(
```

```

        path: '/',
        builder: (_, __) => const HomeScreen(),
      ),
    ],
  );
});

```

10. App Entry Widget

lib/app/nox_app.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../core/theme/nox_theme.dart';
import 'router.dart';

class NoxApp extends ConsumerWidget {
  const NoxApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(routerProvider);

    return MaterialApp.router(
      debugShowCheckedModeBanner: false,
      theme: noxTheme,
      routerConfig: router,
    );
  }
}

```

11. Event Feature Structure

```

features/events/
├── data/
│   ├── datasources/
│   ├── models/
│   ├── mappers/
│   └── repositories_impl/

```



```
|
|
|— domain/
|   |— entities/
|   |— repositories/
|   |— usecases/
|   |
|   |— presentation/
|       |— controllers/
|       |— screens/
|       |— widgets/
```

12. CI/CD Setup

.github/workflows/flutter_ci.yml

```
name: Flutter CI

on:
  push:
    branches:
      - develop
      - main

  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.22.0'

      - run: flutter pub get

      - run: flutter analyze

      - run: flutter test
```

13. Android Release Pipeline

.github/workflows/android_release.yml

```
name: Android Release

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.22.0'

      - run: flutter pub get

      - run: flutter build appbundle --release
```

14. Supabase Migration Workflow

.github/workflows/supabase.yml

```
name: Supabase Migration

on:
  push:
    paths:
      - 'supabase/migrations/**'

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
```

```
- uses: actions/checkout@v4

- run: npm install -g supabase

- run: supabase db push
  env:
    SUPABASE_ACCESS_TOKEN: ${ secrets.SUPABASE_ACCESS_TOKEN }
```

15. Recommended Git Workflow

```
main      → production
develop   → staging
feature/* → development
hotfix/*  → urgent fixes
```

16. Testing Structure

```
test/
├─ unit/
├─ widget/
└─ integration/
```

17. Production Engineering Rules

Mandatory Rules

NEVER

- Put Supabase calls directly in UI
- Hardcode secrets
- Store logic inside widgets
- Use global mutable state
- Skip migrations
- Push directly to main

ALWAYS

- Use repositories
 - Use DTO/model mapping
 - Use Riverpod providers
 - Write tests for usecases
 - Version database schema
 - Use CI pipelines
-

18. Recommended Future Scaling

Phase 2

- Firebase Crashlytics
- Firebase Analytics
- Push notifications
- Supabase Edge Functions

Phase 3

- Recommendation engine
- Event ranking system
- Venue analytics
- Real-time attendance intelligence

Phase 4

- Microservices backend
 - Kubernetes deployment
 - AI nightlife prediction layer
 - Dynamic pricing engine
-

19. Recommended First Production Tasks

Priority order:

1. Authentication stabilization
2. Event repository implementation
3. Secure QR infrastructure
4. Ticket ownership validation
5. Payment verification backend
6. Pulse realtime architecture

- 7. Analytics instrumentation
 - 8. Promoter dashboard
-

20. Final Engineering Principle

NOX is not just a mobile app.

It is:

- a nightlife infrastructure layer
- a city intelligence platform
- a realtime event economy engine
- a trust-based ticketing network

Your architecture decisions now determine whether NOX becomes:

- a temporary beta app OR
- scalable urban infrastructure.